

11 References & the Copy-Constructor

Hu Sikang
skhu@163.com

Contents



- Reference
- Copy Constructor

11.1 References in C++

- A *reference* (&) is usually used for function **argument lists** and function **return values**.
- When a reference is used as a **function argument**, any modification to the reference *inside* the function will cause changes to the argument *outside* the function.
- If you **return** a reference from a function, you must take the same care as if you return a pointer from a function, avoiding to refer to unknown memory.

11.1.1 References in functions

// Simple C++ references

```
int* f(int* x)
{
    (*x)++;
    // Safe, x is outside
    return x;
}
```

```
int& g(int& x)
{
    x++; // Same effect as in f()
    return x; // Safe
}
```

```
int main() {
    int a = 0, b;
    int *p;
    // Ugly (but explicit)
    p = f(&a);
```

```
// Clean (but hidden)
    b = g(a);
    return 0;
}
```

11.1.2 References with local variable

```
int& h()  
{  
    int q = 10;  
    return q; // Error  
}
```

```
int main() {  
    int &a = h();  
    a++;  
    cout << a << endl;  
    return 0;  
}
```

```
int* h()  
{  
    int q = 20;  
    return &q; // Error  
}
```

```
int main() {  
    int* p;  
    p = h();  
    cout << *p << endl;  
    return 0;  
}
```

11.2 The copy-constructor

Initialize the object with other object of the same class.

```
int x = 10;
```

```
int y(x);
```

```
int z{y};
```

```
class Sample;
```

```
Sample S1;
```

```
Sample S2(S1), S3{S2}; // C++ 11
```

```
Sample S4 = S3;
```

11.2 The copy constructor

The genetic form of copy-constructor is:

```
class Sample  
{  
public:  
    Sample(const Sample&); // copy constructor  
};
```

The executing semantic of copy-constructor is to initialize object member data.

11.2.1 Copy Constructors

(1) initialize object

```
class Point {  
    public :  
        Point ( int xx = 0 , int yy = 0 )  
        { X = xx ; Y = yy ; }  
        Point ( const Point & p ) ; // copy constructor  
        int GetX ( ) { return X ; }  
        int GetY ( ) { return Y ; }  
    private : int X , Y ;  
};  
Point :: Point ( const Point& p)  
{  
    X=p.X; Y=p.Y;  
    cout << "Copy-constructor called." << endl ;  
}
```

```
int main ( ) {  
  
    Point A ( 1 , 2 ) ;  
    //call copy-constructor  
    Point B ( A ) ;  
  
    cout << "B:" << B.GetX ( ) ;  
  
    cout << B.GetY ( ) << endl ;  
  
    return 0 ;  
}
```

11.2.1 Copy Constructors

(1) initialize object

The default one is created by the compiler automatically when a copy- constructor don't be defined by user.

```
class Point {  
    public :  
        Point ( int xx = 0 , int yy = 0 ) { X = xx ; Y = yy ; }  
        int GetX ( ) { return X ; }  
        int GetY ( ) { return Y ; }  
    private : int X , Y ;  
};  
  
int main ( ) {  
    Point A ( 1 , 2 ) ;  
    Point B ( A ) ; //call default copy constructor  
    cout << "B:" << B.GetX ( ) << "," << B.GetY ( ) << endl ;  
    return 0 ;  
}
```

11.2.1 Copy Constructors

(2) When the argument is an object, copy-constructor is called.

```
class Point {
public :
    Point ( int xx = 0 , int yy = 0 ) { X = xx ; Y = yy ; }
    Point ( const Point & p ) ;
    ~ Point ( ) { cout << X << "," << Y << " Object destroyed." << endl ; }
    int GetX ( ) { return X ; }    int GetY ( ) { return Y ; }
private : int X , Y ;
} ;
Point :: Point ( const Point & p)           // call copy constructor
{ X = p.X ; Y = p.Y ; cout << "Copy-constructor called." << endl ; }
void f ( Point p ) { cout << "Funtion:" << p.GetX() << "," << p.GetY() << endl ; }
int main ( )
{ Point A ( 1 , 2 ) ;      f ( A ) ; return 0 ; }
```

11.2.1 Copy Constructors

(3) When the returning type of function is class type, copy constructor is called.

```
class Point {  
    public :  
        Point ( int xx=0, int yy=0 )  
            { X=xx ; Y=yy; cout << "Object constructed." << endl ; }  
        Point ( const Point & p ) ;  
        ~ Point ( ) { cout << X << "," << Y << " Object destroyed." << endl ; }  
        int GetX ( ) { return X ; } int GetY ( ) { return Y ; }  
    private : int X , Y ;  
};  
Point :: Point ( const Point & p )  
{ X= p.X ; Y=p.Y ; cout << "Copy_constructor called." << endl ; }  
Point g ( ) { Point A ( 1 , 2 ) ; return A ; }  
int main ( ) { Point B ; B = g ( ) ; return 0 ; }
```

output:

remark:

Implementation

Analysis

```
class Point {  
    public :  
        Point ( int xx=0, int yy=0 )  
            { X=xx ; Y=yy; cout << "Object constructed. " << endl ; }  
        Point ( const Point & p ) ;  
        ~Point() { cout << X << ", " << Y << " Object destroyed. " << endl ; }  
        int GetX ( ) { return X ; }          int GetY ( ) { return Y ; }  
    private : int X , Y ;  
};  
Point :: Point ( const Point & p )  
{ X= p.X ; Y=p.Y ; cout << "Copy_constructor called. " << endl ; }  
Point g ( ) { Point A ( 1 , 2 ) ; return A ; }  
int main ( ) { Point B ; B = g ( ) ; return 0 ; }
```

Implementation Analysis

output:

Object constructed.

remark:

Create a **B's** object

```
class Point {
public :
    Point ( int xx=0, int yy=0 )
        { X=xx ; Y=yy; cout << "Object constructed." << endl ; }
    Point ( const Point & p ) ;
    ~Point() { cout << X << "," << Y << " Object destroyed." << endl ; }
    int GetX () { return X ; }          int GetY () { return Y ; }
private : int X, Y ;
};

Point :: Point ( const Point & p )
{ X= p.X ; Y=p.Y ; cout << "Copy_constructor called." << endl ; }

Point g () { Point A ( 1 , 2 ) ; return A ; }

int main () { Point B ; B = g () ; return 0 ; }
```

Implementation Analysis

output:

Object constructed.
Object constructed.

remark:

Create a **B's** object

Create a local **A's** object

```
class Point {  
    public :  
        Point ( int xx=0, int yy=0 )  
            { X=xx ; Y=yy; cout << "Object constructed." << endl ; }  
        Point ( const Point & p ) ;  
        ~Point() { cout << X << "," << Y << " Object destroyed." << endl ; }  
        int GetX () { return X ; }          int GetY () { return Y ; }  
    private : int X, Y ;  
};  
Point :: Point ( const Point & p )  
{ X= p.X ; Y=p.Y ; cout << "Copy_constructor called." << endl ; }  
Point g () { Point A ( 1, 2 ) ; return A ; }  
int main () { Point B ; B = g () ; return 0 ; }
```

Implementation Analysis

output:

```
Object constructed.  
Object constructed.  
Copy_constructor call.
```

remark:

```
Create a B's object  
Create a local A's object  
Return an anonymous object
```

```
class Point {  
public :  
    Point ( int xx=0, int yy=0 )  
        { X=xx ; Y=yy; cout << "Object constructed." << endl ; }  
    Point ( const Point & p ) ;  
    ~Point() { cout << X << "," << Y << " Object destroyed." << endl ; }  
    int GetX () { return X ; }          int GetY () { return Y ; }  
private : int X, Y ;  
};  
Point :: Point ( const Point & p )  
{ X= p.X ; Y=p.Y ; cout << "Copy_constructor called." << endl ; }  
Point g () { Point A ( 1, 2 ) ; return A ; }  
int main () { Point B ; B = g () ; return 0 ; }
```

Implementation Analysis

output:

```
Object constructed.  
Object constructed.  
Copy_constructor call.
```

remark:

Create a **B's** object

Create a local **A's** object

Return an anonymous object

```
class Point {  
public :  
    Point ( int xx=0, int yy=0 )  
        { X=xx ; Y=yy; cout << "Object constructed." << endl ; }  
    Point ( const Point & p ) ;  
    ~Point() { cout << X << "," << Y << " Object destroyed." << endl ; }  
    int GetX () { return X ; }          int GetY () { return Y ; }  
private : int X, Y ;  
};  
Point :: Point ( const Point & p )  
{ X= p.X ; Y=p.Y ; cout << "Copy_constructor called." << endl ; }  
Point g () { Point A ( 1 , 2 ) ; return A ; }  
int main () { Point B ; B = g () ;    return 0 ; }
```

Assign to object **B**

Implementation Analysis

```
class Point {
public :
    Point ( int xx=0 , int yy=0 ) { X=xx ; Y=yy ; }
    Point ( const Point & p ) {
        ~Point() { cout << X << ", " << Y << " Object destroyed." << endl ; }
        int GetX () { return X ; }      int GetY () { return Y ; }
    private : int X , Y ;
};

Point :: Point ( const Point & p )
{ X= p.X ; Y=p.Y ; cout << "Copy_constructor called." << endl ; }

Point g () { Point A ( 1 , 2 ) ; return A ; }

int main () { Point B ; B = g () ; return 0 ; }
```

output:

```
Object constructed.
Object constructed.
Copy_constructor call.
1 , 2 Object destroyed.
1 , 2 Object destroyed.
1 , 2 Object destroyed.
```

remark:

```
Create a B's object
Create a local A's object
Return an anonymous object
Release local object A
Release anonymous object
Release object B
```

Assign to object B

11.2.1 Copy Constructors

```
class Point {  
    public :  
        Point ( int xx=0, int yy=0 )  
            { X=xx ; Y=yy; cout << "Object constructed." << endl ; }  
        Point ( const Point & p ) ;  
        ~Point() { cout << X << "," << Y << " Object destroyed." << endl ; }  
        int GetX () { return X ; }          int GetY () { return Y ; }  
    private : int X , Y ;  
};  
Point :: Point ( const Point & p )  
{ X= p.X ; Y=p.Y ; cout << "Copy_constructor called." << endl ; }  
Point g () { Point A ( 1 , 2 ) ; return A ; }  
int main () { Point B ; B = g () ; return 0 ; }
```



```
int main () { Point B = g () ; return 0 ; }
```

11.2.2 Exercise (1)

```
// tpoint.h
class TPoint
{
public:
    TPoint(int x,int y) {X=x;Y=y;}
    TPoint(TPoint& p);           //copy constructor
    ~TPoint() {cout<<"Destructor is called."<<endl;}
    int Xcoord() {return X;}
    int Ycoord() {return Y;}
private:
    int X,Y;
};
TPoint::TPoint(TPoint& p)
{
    X=p.X;
    Y=p.Y;
    cout<<"Copy Constructor is called.\n";
}
```

11.2.2 Exercise (1)

// tpoint.cpp

```
TPoint fun(TPoint Q)  // Pass by data value
{
    cout<<"In fun()! "<<endl;
    int x, y;
    x=Q.Xcoord()+10;
    y=Q.Ycoord()+20;
    TPoint R(x,y);
    return R;          // Return R's data value
}

int main( )
{
    TPoint M(20,35),P(0,0);
    TPoint N(M);  //M is a created object, N is a creating object
    P=fun(N);
    cout<<"P="<<P.Xcoord()<<", "<<P.Ycoord()<<endl;
    return 0;
}
```

11.2.2 Exercise (1)

Output:

Copy Constructor is called.

Copy Constructor is called.

In fun()!

Copy Constructor is called.

Destructor is called.

Destructor is called.

Destructor is called. *// temporary object*

P=30,55

Destructor is called.

Destructor is called.

Destructor is called.

11.2.2 Exercise (2)

```
#include <iostream>
using namespace std;
class Point {
public:
    Point(int xx = 0, int yy = 0)
    {
        X = xx; Y = yy;
    }
    Point(const Point & p);
    Point MoveCopy(int offset);
private: int X, Y;
};
```

**How to avoid calling copy-constructor
in "B = A.MoveCopy(5)"?**

```
Point::Point(const Point& p) {
    X = p.X; Y = p.Y;
    cout << "Copy-constructor is called." << endl;
}

Point Point::MoveCopy(int offset) {
    cout << "\"opertor+\" is called." << endl;
    int posX = X + offset;
    int posY = Y + offset;
    Point Temp(posX, posY);
    return Temp;
}

int main( ) {
    Point A(1, 2);
    Point B = A;
    B = A.MoveCopy(5);
    return 0;
}
```

11.2.3* Move Constructor in C++ 11

```
#include<iostream>
using namespace std;
class Point {
public:
    Point(int xx = 0, int yy = 0)
    { X = xx; Y = yy; cout << "Object constructed." << endl; }
    Point(const Point & p);           // Copy Constructor
    Point(Point&& p)                   // Move Constructor
    {
        X = p.X; Y = p.Y; // Copy
        p.X = 0; p.Y = 0; // Restore initial value
        cout << "Moving Constructor" << endl;
    }
    ~Point() { cout << X << ", " << Y << " Object destroyed." << endl; }
    int GetX() { return X; }          int GetY() { return Y; }
private: int X, Y;
};
```

```
Point::Point(const Point & p)
{
    X = p.X; Y = p.Y;
    cout << "Copy_constructor called." << endl;
}

Point g() { Point A(1, 2); return A; }

int main( )
{
    Point p1(5, 8);
    Point p2 = g(); // Call move constructor
    Point p3 = p1; // Call copy constructor

    return 0;
}
```

Right value reference can be used for:

- Temporary objects;
- Constant / Immediate value;

11.2.3* Move Constructor in C++ 11

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> v1 = {1,2,3,4,5,6};
    cout << v1.size() << endl;           // Size is 6

    vector<int> v2 = std::move(v1); // v1 is clear
    cout << v1.size() << endl;           // Size is 0

    for (int a : v1) cout << "a = " << a << ", ";
    cout << endl;

    for (int a : v2) cout << a << ", ";
    cout << endl;

    return 0;
}
```

Sum up Copy-Constructors

- The Copy Constructor is called when **passing arguments by object**.
- The Copy Constructor is **NOT** called when **passing arguments by references or by pointers**. Because no new object is created.
- The Copy Constructor is called when function **returning an object**.